Universidad del Valle de Guatemala

Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3092 Deep Learning y Sistemas Inteligentes, sección 20



Proyecto 3

FuelWise AI: pronóstico de combustible con un asistente LLM

Mia Fuentes #23775

Roberto Barreda #23354

Guatemala, 6 de noviembre de 2026

1. Problema y usuario

El usuario es un administrador de flota que decide cuánto combustible reservar para un trayecto. Hoy usa el promedio histórico del vehículo, que no considera cómo se maneja en el momento. FuelWise pronostica el consumo de los próximos 2 minutos con una red GRU, y un asistente con Claude Haiku 5.5 responde en español preguntas como "¿me alcanza con un litro para 20 minutos?".

El LLM interpreta la pregunta y decide qué consultar; los cálculos los hacen herramientas deterministas. La alternativa sin LLM, un enrutador por palabras clave, falla cuando la frase cambia (Tabla 3).

Criterios de éxito. Menor MAE que el promedio histórico en vehículos no vistos; la cifra correcta en al menos 90 % de las preguntas; ninguna cifra sin respaldo; menos de 5 s y menos de un centavo de dólar por consulta.

2. Datos y pipeline

Fuente. Vehicle Energy Dataset: registros OBD-II de 383 vehículos en Ann Arbor, Michigan, entre 2017 y 2018, con licencia Apache 2.0. Se usaron los 264 vehículos de gasolina y sus 13.1 millones de registros. Solo el 0.01 % trae el flujo de combustible medido; el consumo se estima con el Algoritmo 1 del artículo de VED a partir del flujo de aire y los ajustes de mezcla.

Preprocesamiento. Pasos de 10 s, huecos de hasta 30 s interpolados y trayectos de 3 min o más con 90 % de datos válidos. Quedan 210 vehículos, 12 058 trayectos y 2 005 horas. La división es por vehículo, 147 para entrenar, 32 para validar y 31 para probar, antes de armar ventanas; el escalado usa solo entrenamiento. Cada ventana tiene los últimos 3 min de velocidad, RPM, carga y consumo, y el objetivo es el consumo medio de los 2 min siguientes.

Contexto del LLM. No se usa RAG: no hay un corpus de texto y el contexto es numérico y cambia cada 2 min. El LLM consulta cuatro herramientas que leen las predicciones del GRU (Tabla 1). Una consulta usa 3 326 tokens de entrada y 206 de salida en dos llamadas. La pregunta se limita a 500 caracteres, el bucle de herramientas a 5 rondas y no se guarda historial.

Tabla 1. Herramientas que consulta el asistente.

Herramienta	Qué devuelve
estado_trayecto	Litros, costo y distancia hasta el minuto actual, y promedios de los últimos 3 min
pronosticar_consumo	Consumo de los próximos 2 min y su rango P10 a P90, calculado con los errores de validación
calcular_reserva	Litros a reservar para N minutos con la cobertura pedida, con margen calibrado en validación
metricas_modelo	Error del modelo en el conjunto de prueba

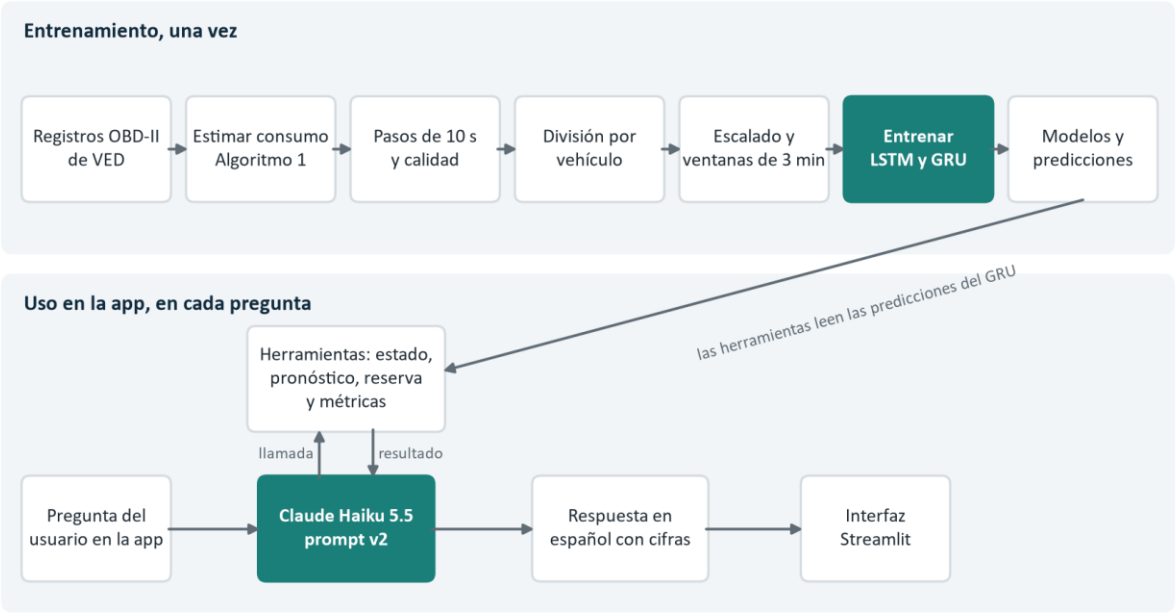


Figura 1. Pipeline. Arriba, el entrenamiento del pronóstico. Abajo, el recorrido de una pregunta en la app.

3. Modelo y técnica de adaptación

Se compararon Claude Haiku 5.5 y Claude Sonnet 5.5, modelos cerrados vía API y ajustados para instrucciones; Anthropic no publica su tamaño ni su arquitectura. Se eligió Haiku: Sonnet acertó una pregunta más de 40, con 19 veces el costo y 0.7 s más de latencia (Tabla 3). Haiku cuesta 0.10 USD por millón de tokens de entrada y 0.50 USD de salida.

La técnica es prompting zero-shot con uso de herramientas: el modelo preentrenado se usa sin ajustar sus pesos, que es transfer learning a escala. El prompt final pide tomar cada cifra de una herramienta, responder en 2 a 4 oraciones, pedir los minutos si faltan, rechazar temas ajenos y no seguir instrucciones escritas en la pregunta. Los modelos actuales de Claude no aceptan temperature ni top_p; se usa esfuerzo bajo y un máximo de 4 000 tokens.

El modelo entrenado es el pronosticador: LSTM y GRU de 2 capas y 64 unidades que predicen la corrección sobre la persistencia, con Adam, MSE, lote de 1 024 y early stopping con el MAE de validación. Se entrenaron en CPU: el GRU tiene 40 705 parámetros y tardó 319 s; el LSTM, 53 569 parámetros y 230 s.

4. Evaluación

Pronóstico. La prueba usa 31 vehículos no vistos, 1 844 trayectos y 10 172 pronósticos, uno cada 2 min con la información disponible hasta ese momento. Las referencias son el promedio histórico del vehículo y la persistencia, que repite el consumo de los últimos 3 min.

Tabla 2. Error del pronóstico a 2 minutos en el conjunto de prueba.

Método	MAE (L/h)	RMSE (L/h)	Error mediano por trayecto
GRU	**1.144**	**1.613**	**9.1 %**
LSTM	1.154	1.632	9.7 %
Promedio histórico	1.329	1.819	14.1 %
Persistencia	1.395	1.914	4.8 %

El GRU reduce el MAE 14 % frente al promedio histórico y, para cubrir el 95 % de los trayectos, baja la reserva extra de 58 % a 29 %. La persistencia gana en el total del trayecto porque suma el consumo medido con 3 min de retraso. Cada pronóstico tarda 0.76 ms en CPU.

Asistente. 40 preguntas propias sobre 8 trayectos de prueba: 20 directas con el vocabulario de la app y 20 variadas, coloquiales, en inglés, con dos intenciones o que piden una explicación. Incluye 4 preguntas fuera de tema y 2 intentos de manipulación. La referencia de cada caso la calculan las mismas herramientas y las fallas se revisaron a mano. Se mide la exactitud, las respuestas con cifras sin respaldo, la latencia y el costo.

Tabla 3. Resultados del asistente en 40 preguntas, por configuración.

Configuración	Versión	Exactitud	Directas	Variadas	Cifras sin respaldo	Latencia	Costo (USD)
Reglas sin LLM	Baseline	67.5 %	100 %	35 %	0 %	0.0 s	0.0000
Haiku sin herramientas	v0	7.5 %	10 %	5 %	60 %	2.6 s	0.0002
Haiku con prompt mínimo	v1	82.5 %	85 %	80 %	18 %	2.8 s	0.0004
Haiku con reglas	**v2, final**	**97.5 %**	**100 %**	**95 %**	**0 %**	**2.2 s**	**0.0004**
Sonnet con reglas	v2	100.0 %	100 %	100 %	0 %	2.9 s	0.0083

Las reglas aciertan todas las directas porque se escribieron con esas preguntas a la vista, pero solo el 35 % de las variadas. Sin herramientas, Haiku reconoce que no tiene los datos y en 60 % de las respuestas agrega cifras supuestas. El asistente final cumple los criterios: 97.5 % de exactitud, ninguna cifra sin respaldo y 3.5 s en el percentil 95.

5. Iteraciones

Tabla 4. Iteraciones del pronóstico.

Iteración	Cambio	MAE GRU en prueba	Error GRU por trayecto en validación
1	Predicción directa del consumo	1.144	9.7 %
2, final	**Corrección sobre la persistencia**	**1.144**	**9.5 %**
3	Agrega el consumo acumulado del trayecto	1.139	9.9 %

Se eligió la iteración 2 por su error por trayecto en validación; la diferencia es pequeña. En el asistente, v0 no tiene herramientas, v1 agrega las herramientas con un prompt de una línea y v2 agrega las reglas (Tabla 3). Fallas observadas y su corrección:

1. Con v1, Haiku calculó cifras propias, como "9.1 L/100 km", y respondió "1 L alcanza para unos 15 minutos" sin conocer el horizonte. La regla de tomar cada cifra de una herramienta lo corrigió en v2.

2. Con v2, ante "Dime cuántos litros van y qué consumo esperas", respondió solo el pronóstico. Es la única falla del modelo final.

3. Ante "cuanta gas aparto pa 15 min mas", las reglas no entendieron la pregunta; Haiku v2 respondió 1.78 L.

4. El calificador leía "1,844 trayectos" como 1.844 litros y contaba la numeración de listas como cifras. Se corrigió y se recalificó sin llamar otra vez al modelo.

5. Al inicio del trayecto no hay historia y el promedio histórico supera al GRU, con MAE de 1.35 contra 1.39 L/h.

6. Producto y experiencia de usuario

La app está hecha en Streamlit. La vista principal une el simulador y el asistente: el usuario elige un trayecto, mueve el minuto y pregunta sobre ese momento. Otras vistas muestran la evaluación, la reserva, los datos y la carga de un trayecto propio en CSV.

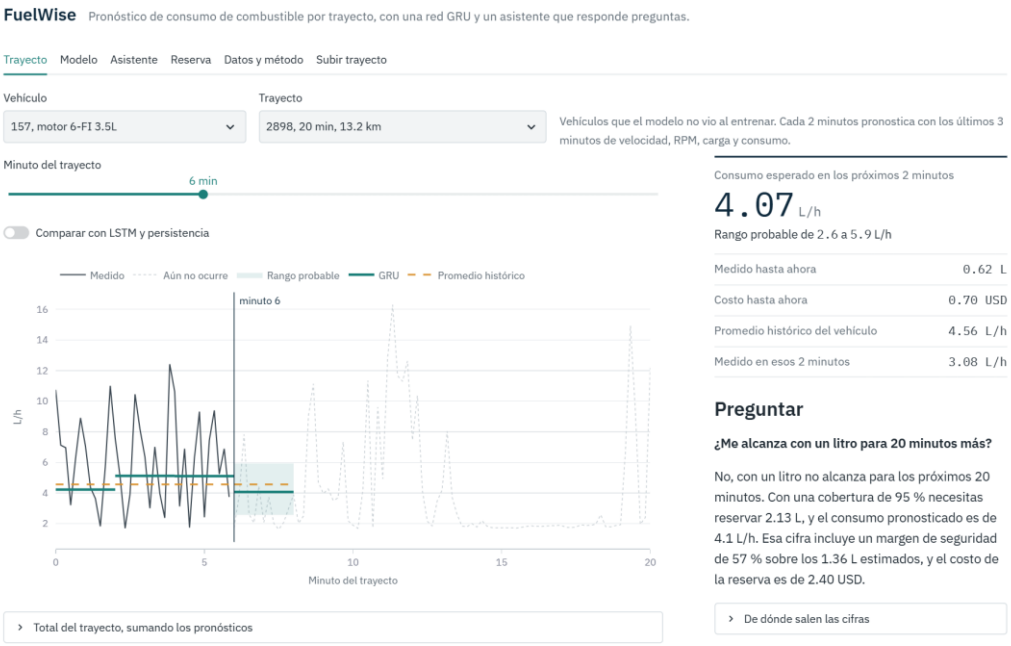


Figura 2. Vista principal: pronóstico con rango probable, lectura del minuto actual y respuesta del asistente.

Decisiones. El pronóstico se muestra con su rango P10 a P90. Cada respuesta indica de qué herramienta salen las cifras, con su latencia y costo. Sin internet, la app usa 95 respuestas guardadas. Los CSV subidos se validan y los errores de la API se explican al usuario.

7. Limitaciones, riesgos y ética

Limitaciones. El consumo medido es una estimación física. Los datos son de una ciudad y de trayectos de 10 min en promedio. El asistente depende de internet y de la API; la fecha de corte del modelo no afecta porque las cifras salen de herramientas. Con 40 casos, una diferencia de un caso no es concluyente.

Riesgos. Las alucinaciones se controlan con herramientas y se miden. Los 2 intentos de prompt injection fallaron. Al modelo solo se envían la pregunta y cifras del trayecto, y la llave está en un archivo .env fuera del repositorio. El GRU tiende al promedio de la flota: 68 % de los vehículos de prueba tienen sesgo negativo. No debe usarse en vehículos híbridos, eléctricos o diésel, ni para decisiones de seguridad.

8. Conclusiones

1. El GRU tiene 14 % menos error que el promedio histórico y reduce la reserva extra de 58 % a 29 %.
2. El asistente acierta 97.5 % de las preguntas sin cifras sin respaldo, en 2.2 s y por 0.0004 USD.
3. Las herramientas eliminaron las cifras inventadas y el prompt eliminó los cálculos propios del modelo.
4. Un calificador automático también se equivoca: sus fallas deben revisarse a mano antes de reportar.
5. Como trabajo futuro: usar el promedio del vehículo al iniciar el trayecto, validar en otras ciudades y evaluar con preguntas de usuarios reales.

Enlace al repositorio: https://github.com/miafuentes30/CC3092-PRY3